

Data Visualization

Week - 8

Timeline Visualization Using D3.js and Google Charts

Lab Objectives

By the end of this lab, students should be able to:

1. Explain the purpose of a timeline visualization.
2. Identify the required fields in a timeline dataset.
3. parse and represent dates in JavaScript.
4. Create a customized event timeline using D3.js.
5. Explain the basic structure of Google Charts.
6. Add Google Charts to a web project.
7. Create a project timeline using Google Charts.
8. Compare D3.js and Google Charts timeline visualizations.

Introduction to Timeline Visualization

A **timeline visualization** displays events or activities according to their dates or time periods.

The horizontal axis normally represents time, while events, activities, or milestones are placed along the axis.

A timeline can show:

- Historical events
- Research milestones
- Project activities
- Product development
- Academic schedules
- Patient records
- Software development stages
- Company history

Two common timeline types

Event timeline

An event timeline displays events that occurred on specific dates.

Example:

Date	Event
10 January 2026	Project Started
5 February 2026	Dataset Collected
20 March 2026	Model Developed
15 April 2026	Report Submitted

Duration timeline

A duration timeline shows when an activity starts and ends.

Activity	Start date	End date
Literature Review	1 January	31 January
Data Collection	1 February	28 February
Model Development	1 March	31 March
Report Writing	1 April	30 April

Duration timelines are similar to Gantt charts.

Required Software

Students need:

- Visual Studio Code
- A web browser
- VS Code Live Server extension
- Internet connection for loading the D3.js and Google Charts libraries

Project Folder

Create a folder named:

timeline-lab

Inside it, create two subfolders:

```
timeline-lab/
├── d3-timeline/
│   ├── index.html
│   ├── style.css
│   └── script.js
└── google-timeline/
    ├── index.html
    ├── style.css
    └── script.js
```

Practical 1: Event Timeline Using D3.js

D3.js HTML File

Open:

d3-timeline/index.html

Add the following code:

```
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>D3.js Project Timeline</title>

  <!-- Load CSS -->
  <link rel="stylesheet" href="style.css">

  <!-- Load the D3.js library -->
  <script src="https://cdn.jsdelivr.net/npm/d3@7"></script>
</head>
```

```

<body>

  <main class="container">
    <h1>University Project Timeline</h1>

    <p class="instruction">
      Move the mouse over a timeline point to view its details.
    </p>

    <div id="timeline"></div>

    <div id="tooltip" class="tooltip"></div>
  </main>

  <script src="script.js"></script>

</body>

</html>

```

D3.js CSS File

Open:

d3-timeline/style.css

Add:

```

/* Reset */
* {
  box-sizing: border-box;
}

/* Body */
body {
  margin: 0;
  padding: 30px;
  font-family: Arial, sans-serif;
  background-color: #f4f7fb;
  color: #1f2937;
}

/* Main Container */
.container {
  max-width: 1100px;

```

```
    margin: auto;
    padding: 25px;
    background-color: white;
    border-radius: 12px;
    box-shadow: 0 4px 15px rgba(0, 0, 0, 0.08);
}

/* Heading */
h1 {
    margin-top: 0;
    text-align: center;
}

/* Instruction Text */
.instruction {
    text-align: center;
    color: #5b6472;
}

/* Timeline Container */
#timeline {
    width: 100%;
    overflow-x: auto;
}

/* Timeline Main Line */
.timeline-line {
    stroke: #6b7280;
    stroke-width: 4;
}

/* Vertical Event Lines */
.event-line {
    stroke: #9ca3af;
    stroke-width: 2;
}

/* Event Circles */
.event-circle {
    cursor: pointer;
    stroke: white;
    stroke-width: 3;
}

/* Event Labels */
```

```
.event-label {
    font-size: 13px;
    font-weight: bold;
    text-anchor: middle;
}

/* Date Labels */
.date-label {
    font-size: 12px;
    fill: #4b5563;
    text-anchor: middle;
}

/* Axis */
.axis text {
    font-size: 12px;
}

.axis path,
.axis line {
    stroke: #9ca3af;
}

/* Tooltip */
.tooltip {
    position: absolute;
    display: none;
    max-width: 260px;
    padding: 12px;
    background-color: #111827;
    color: white;
    border-radius: 7px;
    font-size: 13px;
    line-height: 1.5;
    pointer-events: none;
    box-shadow: 0 4px 10px rgba(0, 0, 0, 0.25);
}

/* Tooltip Heading */
.tooltip strong {
    font-size: 14px;
}
```

D3.js JavaScript File

Open:

d3-timeline/script.js

Add:

```
// -----  
// 1. Timeline Dataset  
// -----  
const timelineData = [  
  {  
    date: "2026-01-10",  
    event: "Project Started",  
    category: "Planning",  
    description: "The project topic and objectives were finalized."  
  },  
  {  
    date: "2026-02-05",  
    event: "Dataset Collected",  
    category: "Data",  
    description: "The required dataset was collected and inspected."  
  },  
  {  
    date: "2026-02-25",  
    event: "Data Cleaned",  
    category: "Data",  
    description: "Missing and incorrect values were handled."  
  },  
  {  
    date: "2026-03-20",  
    event: "Model Developed",  
    category: "Development",  
    description: "The first version of the model was completed."  
  },  
  {  
    date: "2026-04-10",  
    event: "Model Evaluated",  
    category: "Evaluation",  
    description: "The model was tested using several evaluation metrics."  
  },  
  {  
    date: "2026-05-01",  
    event: "Report Submitted",  
    category: "Submission",
```

```

        description: "The final project report was submitted."
    }
];

// -----
// 2. Convert Date Strings into JavaScript Date Objects
// -----
const parseDate = d3.timeParse("%Y-%m-%d");

timelineData.forEach(item => {
    item.date = parseDate(item.date);
});

// -----
// 3. Set Chart Dimensions
// -----
const width = 1000;
const height = 430;

const margin = {
    top: 80,
    right: 70,
    bottom: 80,
    left: 70
};

const timelineY = 220;

// -----
// 4. Create the SVG
// -----
const svg = d3
    .select("#timeline")
    .append("svg")
    .attr("viewBox", `0 0 ${width} ${height}`)
    .attr("width", "100%")
    .attr("height", height);

// -----
// 5. Create the Time Scale
// -----
const xScale = d3
    .scaleTime()
    .domain(d3.extent(timelineData, d => d.date))
    .range([margin.left, width - margin.right]);

```



```

// -----
// 6. Create and Display the Time Axis
// -----
const xAxis = d3
  .axisBottom(xScale)
  .ticks(d3.timeMonth.every(1))
  .tickFormat(d3.timeFormat("%b %Y"));

svg.append("g")
  .attr("class", "axis")
  .attr("transform", `translate(0, ${timelineY + 80})`)
  .call(xAxis);

// -----
// 7. Draw the Main Timeline Line
// -----
svg.append("line")
  .attr("class", "timeline-line")
  .attr("x1", margin.left)
  .attr("x2", width - margin.right)
  .attr("y1", timelineY)
  .attr("y2", timelineY);

// -----
// 8. Create a Color Scale
// -----
const colorScale = d3
  .scaleOrdinal()
  .domain([
    "Planning",
    "Data",
    "Development",
    "Evaluation",
    "Submission"
  ])
  .range([
    "#2563eb",
    "#059669",
    "#7c3aed",
    "#ea580c",
    "#dc2626"
  ]);

// -----

```

```

// 9. Select the Tooltip
// -----
const tooltip = d3.select("#tooltip");

// -----
// 10. Create a Group for Each Timeline Event
// -----
const events = svg
  .selectAll(".event")
  .data(timelineData)
  .enter()
  .append("g")
  .attr("class", "event")
  .attr(
    "transform",
    d => `translate(${xScale(d.date)}, ${timelineY})`
  );

// -----
// 11. Draw Vertical Connector Lines
// Alternate Labels Above and Below the Timeline
// -----
events.append("line")
  .attr("class", "event-line")
  .attr("x1", 0)
  .attr("x2", 0)
  .attr("y1", 0)
  .attr(
    "y2",
    (d, index) => index % 2 === 0 ? -75 : 75
  );

// -----
// 12. Draw Event Circles
// -----
events.append("circle")
  .attr("class", "event-circle")
  .attr("r", 0)
  .attr("fill", d => colorScale(d.category))

  .on("mouseover", function (event, d) {

    d3.select(this)
      .transition()
      .duration(150)

```

```

        .attr("r", 12);

    tooltip
        .style("display", "block")
        .html(`
            <strong>${d.event}</strong><br>
            Date: ${d3.timeFormat("%d %B %Y")(d.date)}<br>
            Category: ${d.category}<br>
            ${d.description}
        `);
    })

    .on("mousemove", function (event) {

        tooltip
            .style("left", `${event.pageX + 15}px`)
            .style("top", `${event.pageY + 15}px`);
    })

    .on("mouseout", function () {

        d3.select(this)
            .transition()
            .duration(150)
            .attr("r", 9);

        tooltip.style("display", "none");
    })

    .transition()
    .duration(700)
    .attr("r", 9);

// -----
// 13. Add Event Names
// -----
events.append("text")
    .attr("class", "event-label")
    .attr("x", 0)
    .attr(
        "y",
        (d, index) => index % 2 === 0 ? -92 : 105
    )
    .text(d => d.event);

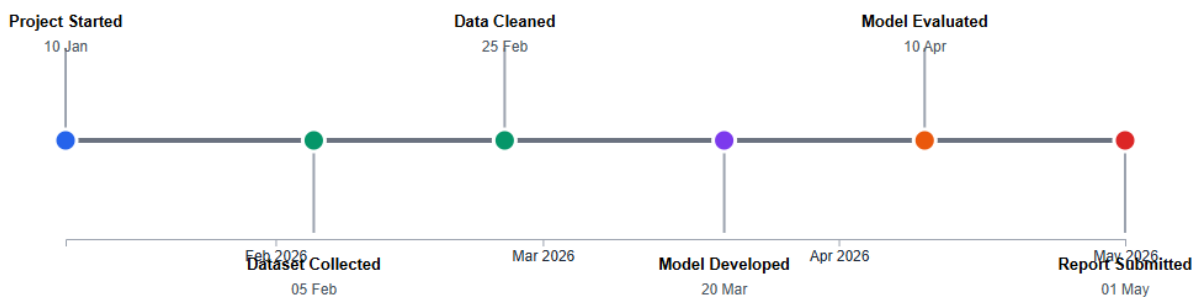
```

```
// -----
// 14. Add Date Labels
// -----
events.append("text")
  .attr("class", "date-label")
  .attr("x", 0)
  .attr(
    "y",
    (d, index) => index % 2 === 0 ? -72 : 125
  )
  .text(d => d3.timeFormat("%d %b")(d.date));
```

Output

University Project Timeline

Move the mouse over a timeline point to view its details.



Important D3.js Concepts Used

`d3.timeParse()`

Converts a date string into a JavaScript date.

```
const parseDate = d3.timeParse("%Y-%m-%d");
d3.scaleTime()
```

Maps dates to positions on the screen.

```
const xScale = d3.scaleTime()
  .domain(d3.extent(data, d => d.date))
  .range([70, 930]);
d3.extent()
```

Finds the minimum and maximum dates.

```
d3.extent(timelineData, d => d.date)
d3.axisBottom()
```

Creates a horizontal time axis.

```
const xAxis = d3.axisBottom(xScale);
```

Data binding

Connects each timeline object to an SVG group.

```
svg.selectAll(".event")
  .data(timelineData)
  .enter()
  .append("g");
```

Introduction to Google Charts

What Is Google Charts?

Google Charts is a JavaScript-based charting library used to create interactive charts for websites.

It provides ready-made visualizations such as:

- Line charts
- Bar charts
- Pie charts
- Column charts
- Scatter plots
- Geo charts
- Sankey diagrams
- Timelines
- Gantt charts
- Tables

Google Charts provides chart constructors, data-table objects and standard methods such as `draw()`. Charts are created through the `google.visualization` namespace.

Unlike D3.js, Google Charts normally requires less code because the visual structure of the chart is already implemented.

Advantages

- Easy to learn
- Requires less code
- Includes interactive tooltips
- Includes several ready-made chart types
- Suitable for dashboards
- Supports responsive browser-based visualization
- Accepts JavaScript data tables

How Google Charts Works

A Google Charts project usually follows six steps.

Step 1: Load the Google Charts library

Add this inside the HTML `<head>`:

```
<script src="https://www.gstatic.com/charts/loader.js"></script>
```

Google's documentation uses this loader to load chart libraries and packages.

Step 2: Load the required chart package

For a timeline, load the timeline package:

```
google.charts.load("current", {  
  packages: ["timeline"]  
});
```

Step 3: Set the callback function

The callback runs after the library has loaded.

```
google.charts.setOnLoadCallback(drawChart);
```

Step 4: Create a chart container

```
<div id="timeline-chart"></div>
```

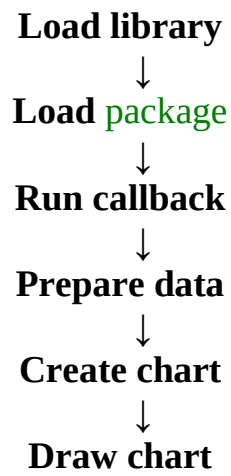
Step 5: Create the data table

```
const dataTable = new google.visualization.DataTable();
```

Step 6: Create and draw the chart

```
const chart = new google.visualization.Timeline(  
  document.getElementById("timeline-chart")  
);  
chart.draw(dataTable, options);
```

Therefore, the general structure is:



Practical 2: Research Project Timeline Using Google Charts

Google Charts HTML File

Open:

google-timeline/index.html

Add:

```
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">

  <meta
    name="viewport"
    content="width=device-width, initial-scale=1.0"
  >

  <title>Google Charts Research Timeline</title>

  <!-- Load CSS -->
  <link rel="stylesheet" href="style.css">

  <!-- Load Google Charts -->
```

```

    <script src="https://www.gstatic.com/charts/loader.js"></script>
</head>

<body>

    <main class="container">
        <h1>Research Project Timeline</h1>

        <p>
            This timeline shows the planned duration of each research activity.
        </p>

        <div id="timeline-chart"></div>
    </main>

    <!-- Load JavaScript -->
    <script src="script.js"></script>

</body>

</html>

```

Google Charts CSS File

Open:

google-timeline/style.css

Add:

```

/* =====
   Global Styles
===== */
* {
    box-sizing: border-box;
}

/* =====
   Body
===== */
body {
    margin: 0;
    padding: 30px;
    font-family: Arial, sans-serif;
    background-color: #eef2f7;
    color: #1f2937;
}

```



```
}

/* =====
   Main Container
===== */
.container {
  max-width: 1100px;
  margin: auto;
  padding: 25px;
  background-color: white;
  border-radius: 12px;
  box-shadow: 0 4px 15px rgba(0, 0, 0, 0.08);
}

/* =====
   Heading
===== */
h1 {
  margin-top: 0;
  text-align: center;
}

/* =====
   Description
===== */
p {
  text-align: center;
  color: #4b5563;
}

/* =====
   Timeline Chart
===== */
#timeline-chart {
  width: 100%;
  height: 460px;
  margin-top: 25px;
}
```

Google Charts JavaScript File

Open:

google-timeline/script.js

Add:

```
// -----  
// 1. Load the Google Charts Timeline Package  
// -----  
google.charts.load("current", {  
    packages: ["timeline"]  
});  
  
// -----  
// 2. Run drawChart() After the Library Loads  
// -----  
google.charts.setOnLoadCallback(drawChart);  
  
// -----  
// 3. Create the Timeline Chart  
// -----  
function drawChart() {  
  
    // Find the HTML container  
    const container = document.getElementById("timeline-chart");  
  
    // Create the Google Timeline chart  
    const chart = new google.visualization.Timeline(container);  
  
    // Create an empty data table  
    const dataTable = new google.visualization.DataTable();  
  
    // -----  
    // Define Table Columns  
    // -----  
    dataTable.addColumn({ type: "string", id: "Phase" });  
    dataTable.addColumn({ type: "string", id: "Activity" });  
    dataTable.addColumn({ type: "date", id: "Start" });  
    dataTable.addColumn({ type: "date", id: "End" });  
  
    // -----  
    // Add Research Project Activities  
    // -----  
    dataTable.addRow([
```

```
[
    "Planning",
    "Topic Selection",
    new Date(2026, 0, 1),
    new Date(2026, 0, 15)
],

[
    "Planning",
    "Proposal Development",
    new Date(2026, 0, 16),
    new Date(2026, 0, 31)
],

[
    "Literature",
    "Literature Review",
    new Date(2026, 0, 10),
    new Date(2026, 1, 28)
],

[
    "Data",
    "Dataset Collection",
    new Date(2026, 1, 1),
    new Date(2026, 1, 20)
],

[
    "Data",
    "Data Preprocessing",
    new Date(2026, 1, 21),
    new Date(2026, 2, 15)
],

[
    "Development",
    "Model Development",
    new Date(2026, 2, 10),
    new Date(2026, 3, 15)
],

[
    "Evaluation",
```

```

        "Model Evaluation",
        new Date(2026, 3, 16),
        new Date(2026, 4, 5)
    ],

    [
        "Writing",
        "Report Writing",
        new Date(2026, 3, 20),
        new Date(2026, 4, 20)
    ],

    [
        "Submission",
        "Final Submission",
        new Date(2026, 4, 21),
        new Date(2026, 4, 28)
    ]
]);

// -----
// Timeline Chart Configuration
// -----
const options = {

    timeline: {

        groupByRowLabel: true,
        showRowLabels: true,
        showBarLabels: true,

        rowLabelStyle: {
            fontName: "Arial",
            fontSize: 14
        },

        barLabelStyle: {
            fontName: "Arial",
            fontSize: 12
        }
    },

    avoidOverlappingGridLines: false
};

```

```

// -----
// Draw the Timeline Chart
// -----
chart.draw(dataTable, options);
}

// -----
// 4. Redraw the Chart When the Window is Resized
// -----
window.addEventListener("resize", drawChart);

```

Understanding JavaScript Dates

JavaScript counts months from **0 to 11**.

Month	JavaScript number
January	0
February	1
March	2
April	3
May	4
June	5
July	6
August	7
September	8
October	9
November	10
December	11

Therefore:

`new Date(2026, 0, 1)`

Means:

1 January 2026

This is a common source of errors in Google Charts projects.

Understanding the Google Timeline Columns

The practical uses four columns.

```
dataTable.addColumn({  
  type: "string",  
  id: "Phase"  
});
```

This identifies the row or project phase.

```
dataTable.addColumn({  
  type: "string",  
  id: "Activity"  
});
```

This contains the activity name displayed inside the bar.

```
dataTable.addColumn({  
  type: "date",  
  id: "Start"  
});
```

This contains the starting date.

```
dataTable.addColumn({  
  type: "date",  
  id: "End"  
});
```

This contains the ending date.

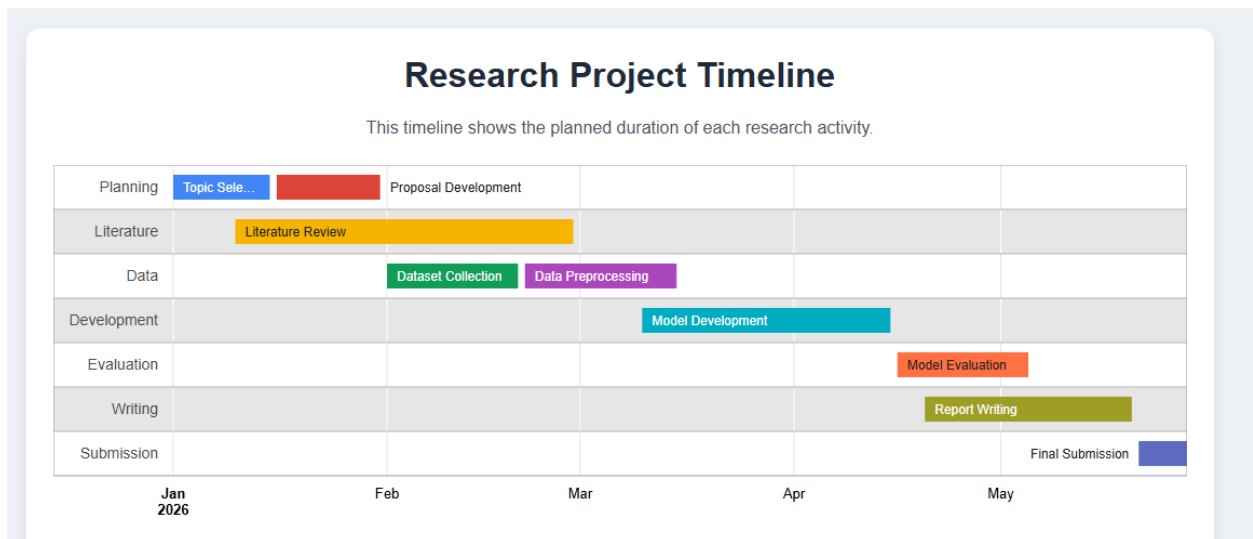
Each row therefore follows this structure:

```
[  
  "Phase",  
  "Activity",  
  startDate,  
  endDate  
]
```

Example:

```
[  
  "Development",  
  "Model Development",  
  new Date(2026, 2, 10),  
  new Date(2026, 3, 15)  
]
```

Output



Output in Mobile and Tab view



Assignment: Interactive Historical Timeline using D3.js

Objective

The objective of this assignment is to assess your understanding of **D3.js**, **SVG**, **data binding**, **CSV data loading**, **time scales**, and **interactive visualization** by developing an interactive historical timeline using a real-world dataset.

Scenario

You have been hired by a digital history museum to build an interactive web application that visualizes important historical events from around the world. Visitors should be able to explore events chronologically, search for specific incidents, and filter events based on different categories.

Dataset

Dataset Name: World Important Dates

(Link: <https://www.kaggle.com/datasets/saketk511/world-important-events-ancient-to-modern?resource=download>)

The dataset contains information such as:

- Date
- Month
- Year
- Name of Incident
- Country
- Place Name
- Type of Event
- Outcome
- Impact
- Affected Population
- Important Person/Group Responsible

Task 1: Create an Interactive Timeline

Develop a horizontal timeline where each historical event is represented as a circle positioned according to its year.

The timeline should include:

- Horizontal timeline axis
- Year labels
- Event markers
- Proper spacing between events
- Responsive SVG layout

Task 2: Load Data from CSV

Load the provided CSV file using **D3.js**.

Your visualization must:

- Read data directly from the CSV file
- Display all valid historical events
- Handle missing or unknown values appropriately

Task 3: Event Details (Tooltip)

When the user hovers over an event, display a tooltip containing at least:

- Event name
- Date
- Country
- Event type
- Outcome

You may include additional information if desired.

Task 4: Search Functionality

Implement a search box that allows users to search events by their incident name.

The timeline should update dynamically while typing.

Task 5: Filtering

Create filter controls for **any TWO** of the following:

- Country
- Event Type
- Outcome

Selecting a filter should immediately update the timeline.

Task 6: Color Encoding

Use different colors to represent event categories.

You may choose one of the following attributes:

- Outcome
- Event Type
- Country

Include a legend explaining the colors used.

Task 7: Summary Dashboard

Display at least **three** summary statistics above the timeline.

Examples include:

- Total visible events
- Earliest event year
- Latest event year
- Number of countries represented
- Number of event types

Expected Learning Outcomes

HISTORICAL DATA VISUALIZATION

World Important Dates Timeline

Explore historical events across countries, event types, outcomes, and time.

Search incident

Type an incident name...

Country

All countries

▼

Event type

All event types

▼

Outcome

All outcomes

▼

Reset filters

Reset filters

Visible events

1,096

Earliest year

2600 BCE

Latest year

2022 CE

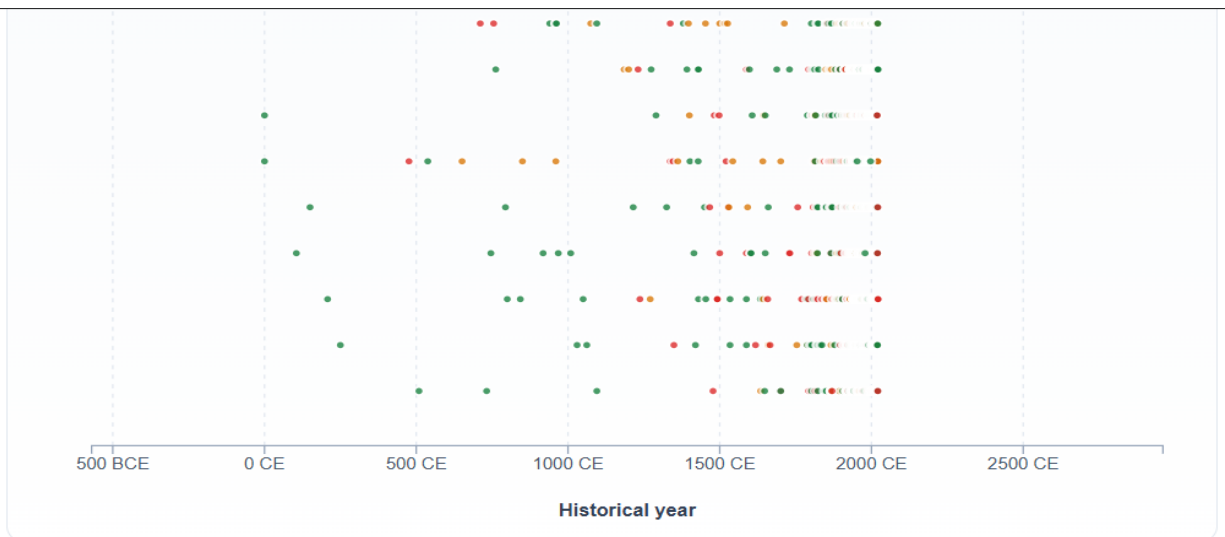
Countries shown

113

Interactive historical timeline

Scroll to zoom, drag to pan, and hover over a point for details.

● Positive ● Negative ● Mixed ● Ongoing



Many events are visible. Zoom in to separate events that occur in similar years.

Submission Requirements

Submit **one ZIP file** containing:

- index.html
- style.css
- script.js
- World Important Dates.csv
- Screenshots of the final output